

Laboratori de GIA-ABIA

Sessió 2: Depuració, generadors i estat com a objecte

Depuració

Depurar és una habilitat essencial per a qualsevol desenvolupador de software. Consisteix en identificar i corregir errors o anomalies en el codi, de manera interactiva i pausant, si és necessari, el fluxe d'execució. En aquesta sessió, provarem a depurar el codi del Problema 5, que ja tenim codificat en el fitxer `problema_5.py`.

Aprendre a identificar i resoldre problemes en el codi no només millora la qualitat del codi, sinó que també us pot ajudar a comprendre millor com funcionen els programes i llibreries en profunditat.

Recordeu que un dels exercicis proposats a la Sessió 1 era reemplaçar la crida a `breadth_first_tree_search` per `depth_first_tree_search` (DFS sense gestió de repetits). Quan executeu el codi, no us dona cap output i, a més, no acaba mai. Utilitzarem un depurador per entendre què està passant.

Farem servir `pdb` (*Python debugger*) que està inclòs en totes les distribucions de Python. Alternativament, podeu utilitzar PyCharm Community Edition o Visual Studio Code amb plugins específics però els haureu d'instal·lar, configurar i operar pel vostre compte.

La metodologia que seguirem serà:

1. **Afegir punts de trencament:** Col·locarem un punt de trencament a la línia de codi a on voldrem parar l'execució. Podria ser qualsevol, però ja que el programa no acaba i podríem intuir que s'ha ficat en un bucle infinit, buscarem un punt que sabem que potencialment s'executarà molts cops.
2. **Execució pas a pas:** Una vegada que l'execució s'aturi en el punt de trencament, utilitzarem les eines del depurador per anar pas a pas a través del codi. Això ens permetrà veure amb detall quines operacions s'executen una a una i ens permet intuir com canvia l'estat del programa.
3. **Inspecció de variables:** Durant la depuració, podem fer ús de les eines del depurador per inspeccionar les variables. Això pot ser útil per veure els valors actuals i comprendre millor com funciona el codi.

Depurant el DFS: Què va malament?

L'algoritme de DFS explora tan profundament com sigui possible abans de retrocedir. Sense una gestió adequada dels estats repetits, pot entrar en bucles infinits, especialment en problemes com el Problema 5 a on les configuracions del tauler es poden repetir amb determinades combinacions d'accions. Anem pas a pas.

Primer canviem l'algoritme:

```
1 from aima.search import depth_first_tree_search
2
3 node = depth_first_tree_search(problem)
4 print(node.solution())
```

Com ja hem comentat abans, no arribem mai al `print(node.solution())`. Ara importem la funció que permet al depurador parar l'execució:

```
1 from pdb import set_trace
```

El següent pas és decidir a on col·locar el punt de trencament, és a dir, a on volem que pari l'execució. Pot ser en qualsevol punt que vulgueu i podeu provar a diferents llocs. Nosaltres us proposem de fer-ho just abans de la línia `return current_state == self.goal_state` del mètode `goal_test`, ja que si hi ha un bucle a l'exploració de l'espai d'estats, el més probable és que aquest mètode s'executi continuament (ja que s'executa cada cop que l'algoritme explora un nou estat per comprovar si ja hem arribat al final), i `current_state` és en aquest punt una variable que probablement contindrà informació rellevant que ens ajudi a entendre què està passant. El codi quedaria així:

```
1 set_trace()
2 return current_state == self.goal_state
```

Ara podem executar amb normalitat:

```
1 python problema_5.py
```

I ens sortirà immediatament un output semblant a:

```
1 > .../abia/problema_5.py(48)goal_test()
2 -> return current_state == self.goal_state
3 (Pdb)
```

L'output pot variar depenent del vostre directori i de la línia a on us ha quedat la següent línia després del `set_trace()`. El prompt (Pdb) indica que estem aturats en un punt de trencament (o *breakpoint*) i ara podeu executar comandes de depuració. Les més bàsiques són:

- `n` o `next`: Executa la següent línia de codi.

- `s` o `step`: Entra dins de la funció que es crida en la línia actual.
- `c` o `continue`: Continua l'execució fins al següent breakpoint.
- `q` o `quit`: Surt del depurador.
- `p nom_variable`: Mostra el valor de la variable `nom_variable`.
- `bt`: Mostra la pila de crides, és a dir, l'encadenament de crides a funcions que ens han portat a aquest punt de l'execució.

Hi ha més comandes i les podeu consultar a <https://docs.python.org/3/library/pdb.html#debugger-commands>.

Anem a inspeccionar la variable `current_state` en aquest punt¹:

```
1 (Pdb) p current_state
2 ['A', 'R', 'A', 'R', 'A']
3 (Pdb)
```

Podem deduir que l'algoritme està comprovant si l'estat inicial és final. Per assegurar-nos podem consultar la pila de crides:

```
1 (Pdb) bt
2   .../abia/problema_5.py(56)<module>()
3 -> node = depth_first_tree_search(problem)
4   .../venv/lib/python3.10/site-packages/aima/search.py(210)
      depth_first_tree_search()
5 -> if problem.goal_test(node.state):
6 > .../abia/problema_5.py(48)goal_test()
7 -> return current_state == self.goal_state
8 (Pdb)
```

La pila de crides ens diu que estem a `goal_test` (on hem posat el breakpoint), que s'ha cridat des de `depth_first_tree_search`, que en el seu torn s'ha cridat des del nostre main de `problema_5.py`.

Ara anem a deixar que el programa corri amb normalitat de nou fins que arribi al mateix punt de trencament. A veure què passa:

```
1 (Pdb) c
2 > .../abia/problema_5.py(48)goal_test()
3 -> return current_state == self.goal_state
```

Estem de nou al mateix punt. Inspeccionem `current_state`:

¹Depenent de com tingueu instal·lat Python, és possible que pogueu utilitzar el tabulador per autocompletar noms de variable.

```
1 (Pdb) p current_state
2 ['R', 'R', 'A', 'R', 'R']
3 (Pdb)
```

Sembla que estem en el resultat d'aplicar l'acció 4. És un comportament consistent amb el fet de que aquesta implementació de DFS utilitza una pila per gestionar els nodes oberts. Podeu comprovar aquest comportament depurant pas a pas (entrant a dins amb la comanda `step`) per veure com funciona `frontier.extend()`, però us ho deixem com a exercici si teniu curiositat.

Continuem de nou l'execució i tornem a inspeccionar:

```
1 (Pdb) c
2 > .../abia/problema_5.py(48)goal_test()
3 -> return current_state == self.goal_state
4 (Pdb) p current_state
5 ['A', 'R', 'A', 'R', 'A']
```

Estem de nou a l'estat inicial! Per què? Què passa si tornem a continuar l'execució un cop? I si ho fem més cops?

Us recomanem que agafeu soltura amb el procés de depuració i que practiqueu les comandes de `next` (executar fins la següent línia de codi) i `step` (entrar dins de la crida a funció de la línia actual, si n'hi ha), de cara a la pràctica de laboratori.

Generadors en Python: estalviar memòria en problemes de cerca

Un *generador* és una funció especial en Python que permet iterar sobre un conjunt de valors, però en comptes de retornar tots els valors a la vegada (com faria una llista), un generador els retorna un a un, a mesura que es van necessitant. Això permet estalviar una gran quantitat de memòria, ja que els valors es generen i es consumeixen sobre la marxa, en comptes de ser emmagatzemats de cop en memòria.

Exemple: Problema 5

A la Sessió 1 us vam demanar que utilitzéssiu la classe `Flip` en comptes de tenir enters com a identificadors d'acció. El més probable és que hagueu implementat el mètode `actions()` utilitzant un codi similar a:

```
1 def actions(self, current_state: List[str]) -> List[Flip]:
2     total_coins = len(current_state)
3     all_actions = []
```

```
4     for i in range(total_coins):
5         all_actions.append(Flip(i, (i + 1) % total_coins))
6     return all_actions
```

O, en la seva forma equivalent utilitzant *list comprehensions*:

```
1 def actions(self, current_state: List[str]) -> List[Flip]:
2     total_coins = len(current_state)
3     return [Flip(x, (x + 1) % total_coins) for x in range(
4         total_coins)]
```

Aquestes implementacions són teòricament correctes, però en la pràctica poden suposar inconvenients, especialment en problemes de cerca amb factors de ramificació molt grans. Per exemple, un DFS es podria implementar sense una pila, per visitar només el primer successor de cada estat, i per tant no caldria generar el segon successor fins que hi hagués *backtracking*.

A la llarga i en general, podem estalviar memòria (en cas mig, no en cas pitjor) fent servir generadors.

Sintaxi bàsica d'un generador

Podeu crear un generador fent servir la paraula clau `yield` en comptes de `return` dins d'una funció. Aquí teniu un exemple senzill:

```
1 def my_generator():
2     yield 1
3     yield 2
4     yield 3
5
6 gen = my_generator()
7 print(next(gen)) # Output: 1
8 print(next(gen)) # Output: 2
9 print(next(gen)) # Output: 3
```

Una forma equivalent, utilitzant *comprehensions*, és la següent:

```
1 def my_generator():
2     return (x + 1 for x in range(3))
3
4 gen = my_generator()
5 print(next(gen)) # Output: 1
6 print(next(gen)) # Output: 2
7 print(next(gen)) # Output: 3
```

Compareu aquests dos codis per comprovar la diferència en execució entre utilitzar llistes i utilitzar generadors:

```
1 # (Versio amb llista)
2 def print_and_inc(x):
3     print("Generant: " + str(x + 1))
4     return x + 1
5
6 for i in [print_and_inc(x) for x in range(3)]:
7     print(i)
```

```
1 # (Versio amb generador)
2 def print_and_inc(x):
3     print("Generant: " + str(x + 1))
4     return x + 1
5
6 for i in (print_and_inc(x) for x in range(3)):
7     print(i)
```

A continuació mostrem dos exemples de com modificar la funció `actions()` del Problema 5 per a retornar un generador. Escolliu la que preferiu, ja que són equivalents:

```
1 def actions(self, current_state: List[str]) -> Generator[Flip,
2     None, None]:
3     total_coins = len(current_state)
4     for x in range(total_coins):
5         yield Flip(x, (x + 1) % total_coins)
```

```
1 def actions(self, current_state: List[str]) -> Generator[Flip,
2     None, None]:
3     total_coins = len(current_state)
4     return (Flip(x, (x + 1) % total_coins) for x in range(
5         total_coins))
```

Quanta memòria estalviareu depèn de la implementació dels algoritmes. En general, amb els algoritmes de cerca no informada o informada de la llibreria d'AIMA no guanyareu² però amb alguns de cerca heurística i amb els de cerca

²Són implementacions molt bàsiques i poc optimitzades més pensades per a representar una traducció gairebé literal dels algoritmes explicats al llibre.

local (que són els que utilitzareu en la pràctica) sí que tindreu guanys considerables.

Utilitzant objectes per representar estats

Quan fem servir l'algoritme DFS amb gestió de repetits (`depth_first_graph_search`), ens trobem amb l'error que diu que el tipus de dades de l'estat (una llista) no és *hashable*. Això passa perquè, en Python, les llistes són mutables i, per tant, no poden ser utilitzades com a claus en un diccionari o, com en el cas de la implementació d'AIMA, en un conjunt (estructura de nodes visitats `explored` als algoritmes `_first_graph_search`).

Una solució a aquest problema és encapsular l'estat en un objecte que sigui immutable, garantint que, un cop creat, no pot ser modificat. Això ens permet utilitzar l'objecte com a clau en un diccionari sense problemes. Les classes definides *ad hoc* a Python són immutables per defecte. És altament recomanable que:

- Implementem la classe pel nostre estat de manera immutable, és a dir, que cap mètode modifiqui el seu propi estat i estiguem obligats a crear nous objectes via `copy()` per crear versions modificades d'un mateix objecte. Això ens permet evitar col·lisions o errors de consistència quan utilitzem els objectes de la classe com a índexos d'un diccionari o conjunt.
- Reimplementem els mètodes `__eq__` i `__hash__` per garantir que, quan comparem dos estats amb els mateixos valors (per exemple la llista de monedes en el problema 5), sempre els considerem equivalents. Si no es reimplementen, `__eq__` per defecte sempre retorna fals, i `__hash__` retorna l'adreça de memòria de l'objecte, i per tant no ens serveixen.
- Afegim qualsevol operació auxiliar que pertanyi al domini de l'estat com a mètode de la classe (com feieu a la pràctica de PA2). D'aquesta manera evitem exposar la implementació interna de l'estat cap a altres parts del codi.

Sota aquestes premisses, us recomanem la següent implementació per al Problema 5:

```
1 class ImmutableCoinsState(object):
2     def __init__(self, coins_state: List[str]):
3         self.coins_state = coins_state
4
5     def generate_actions(self) -> Generator[Flip, None, None]:
6         total_coins = len(self.coins_state)
7         return (Flip(x, (x + 1) % total_coins) for x in range(
8             total_coins))
```

```

9     def apply_action(self, next_action: Flip) ->
10         ImmutableCoinsState:
11             new_state = self.coins_state.copy()
12             new_state[next_action.pos_1] = self.flip_coin(self.
13                 coins_state[next_action.pos_1])
14             new_state[next_action.pos_2] = self.flip_coin(self.
15                 coins_state[next_action.pos_2])
16
17             return ImmutableCoinsState(new_state)
18
19     @staticmethod
20     def flip_coin(coin: str):
21         if coin == 'A':
22             return 'R'
23         elif coin == 'R':
24             return 'A'
25         else:
26             raise Exception("Coin is not valid, should be A or
27                 R")
28
29     def length(self):
30         return len(self.coins_state)
31
32     def __eq__(self, other):
33         return isinstance(other, ImmutableCoinsState) and self.
34             coins_state == other.coins_state
35
36     def __hash__(self):
37         return hash(tuple(self.coins_state))
38
39     def __repr__(self):
40         return str.join("", self.coins_state)

```

Analitzeu com s'implementen els mètodes, amb especial atenció a com es reimplementen la funció de hash i la igualtat, i com s'aconsegueix que l'objecte sigui immutable.

Següents passos

De cara a la Sessió 3, haurieu de preparar el següent:

- Adapteu el main de problema_5.py per a que la classe Problema5 utilitzi la implementació proposada de la classe ImmutableCoinsState.
- Adapteu la vostra implementació del Problema 15, per tal que tingui accions i estats com a objectes. Haureu de proposar classes tant pels operadors com per representar l'estat.

- Tant amb el Problema 5 com amb el Problema 15, proveu els algoritmes BFS i DFS amb o sense gestió de repetits, i el de profunditat iterativa: `iterative_deepening_search`.